What do physics foundation models share inside?

Looking inside Walrus: do different PDEs use the same parts of the model, and is what they share physics?

Research plan · Maarten Stork · 28 October 2026

In one paragraph. Foundation models for PDEs are trained once on many kinds of physics and then learn new problems faster. Nobody knows why. One explanation: inside, the model has built something that holds for many equations, a shared core. The other: it keeps every equation in its own corner, and only generic skills carry over, so it is several models in one box. We test this inside *Walrus* [1], a released 1.3-billion-parameter physics foundation model trained on 19 systems. **Part A** asks *whether* different PDEs share the inside of the model, with four tests we built and checked on our own model (neurons, directions, paths, and removing parts). A small *lab model* we can retrain adds what Walrus cannot give: what happens when the model has no room to keep things apart. **Part B**, only where Part A finds sharing, asks *what* is shared: can physical quantities be read out of it, do they agree with each other the way the laws of physics say, and does the model use them? On our own small model (Poseidon-T design, 8 PDEs), Part A already gives a clear first answer: the PDEs use the same neurons, but mostly separately per equation, and not grouped by PDE type. The question now is whether a real, large foundation model does the same.

Contents. 1 The big goal 2 Words we use 3 What we already know 4 Related work 5 The models 6 Part A: is it shared? 7 Part B: is it physics? 8 Order 9 Risks 10 What comes out

1 The big goal

1.1 Foundation models for PDEs work, and nobody knows why

A *foundation model* for PDEs is one large network, trained once on simulations of many kinds of physics: fluids, waves, chemistry, plasma. Afterwards it learns a new problem from far fewer examples than a model trained from scratch. Poseidon [2], Multiple Physics Pretraining [3] and Walrus [1] all show this.

What these papers do not show is *why*. The model never sees an equation, only fields on a grid. So what is it that carries over from one kind of physics to another?

This is the question from our supervisor meeting: *is there some shared "magic" inside, a physical pattern that helps for all these equations?* The whole point of one big model for many kinds of physics rests on it. There are two very different answers (Figure 1).

- **Story 1, shared magic.** Inside, the model has built something that holds for many equations: for example a common way to handle transport, diffusion or waves. Equations that behave alike (the textbook PDE types: elliptic, parabolic, hyperbolic) would then sit together inside. That is what justifies one big model for all physics, and that shared part is worth studying: it might be a picture of physics.

- **Story 2, several models in one box.** Inside, every equation keeps to its own part. Only generic things are shared: reading a field in, drawing one out, keeping numbers in range. Then the case for one big model is weak: it is mainly a convenient package, and a few separate, smaller models would make just as much sense.

We do not set out to show which design predicts better. We look inside, because that is where the justification for one big model either holds or does not.

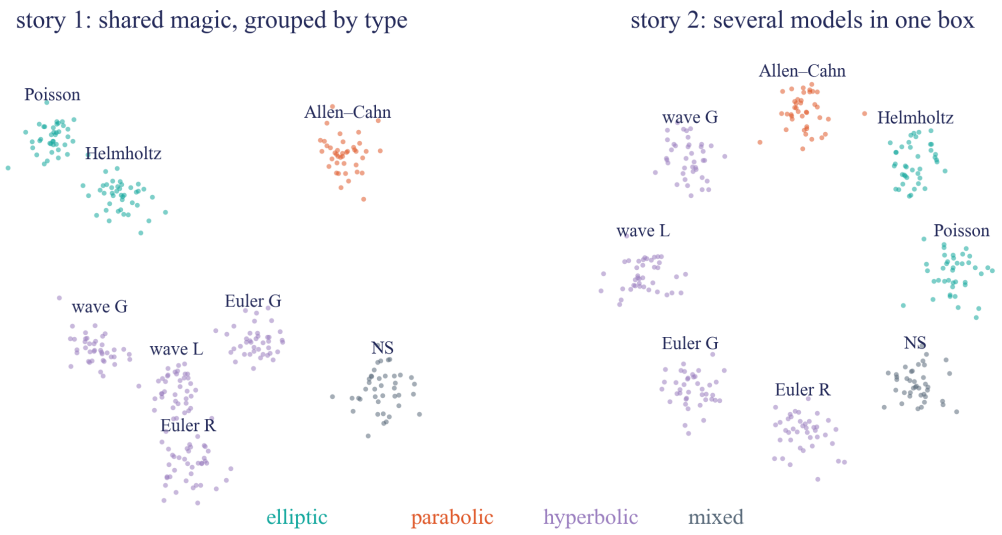


Figure 1: The two stories, as a cartoon (not data). Each cloud is the inside activity of one PDE. Left: PDEs of the same type sit together, shared magic. Right: every PDE has its own corner.

1.2 Why it matters

- **The case for one big model.** If the inside is shared, one model for all physics is well justified. If it is several models in one box, that justification is weak, and separate parts per type of physics, or separate smaller models, make as much sense.

- **Trust.** A model that holds a consistent picture of the physics should behave sensibly on new settings. A model built from separate shortcuts can fail without warning.

- **Understanding transfer.** Pretraining helps; knowing what is shared tells us what kind of pretraining helps, and why.

1.3 The main question

> *Do different PDEs share the inside of a physics foundation model? And where they do, is what they share a picture of the physics?*

The question has two parts, in this order. The second only makes sense where the first finds something.

Part A: is it shared? Do different PDEs use the same parts of the model, and does the model need those parts? Are PDEs of the same type grouped together? Section 6

Part B: is it physics? Only where Part A finds sharing. Can physical quantities be read out of the shared part, do they agree with each other through the laws of physics, and does the model use them? Section 7

Careful: shared is not yet physics. A shared part can be generic: smoothing, taking differences, keeping numbers in range. That is useful for every PDE, but it is not a picture of the physics. Sharing is the precondition; Part B decides. And the other way round: if Part A finds every PDE in its own corner, that is itself the answer (story 2), and an interesting one.

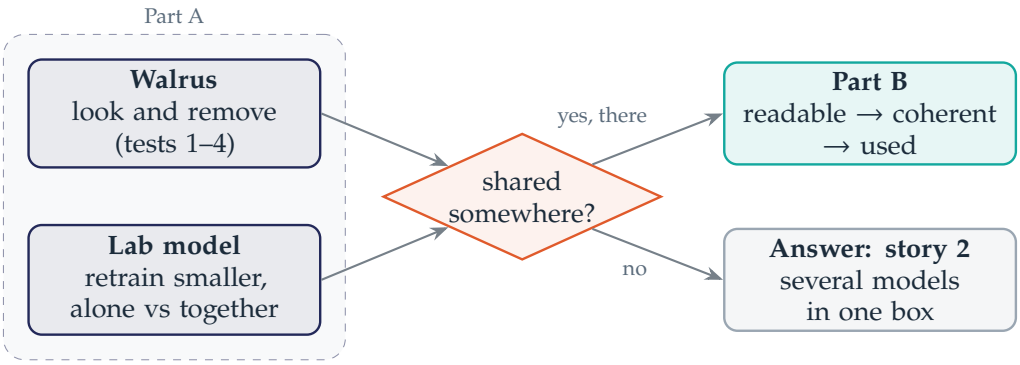


Figure 2: The plan in one picture. Part A looks inside Walrus and experiments on a small lab model. What it finds decides whether, and where, Part B looks for physics.

2 Words we use

PDE type. The textbook sorting of PDEs by how information travels. *Elliptic*: a steady state, every point depends on all others at once (Poisson, Helmholtz). *Parabolic*: spreads and smooths over time (heat, reaction–diffusion). *Hyperbolic*: travels at a finite speed and stays sharp (waves, compressible gas). Some, like the incompressible Navier–Stokes equations, are *mixed*.

System. One dataset: one equation in one setting (for example compressible Euler with periodic walls). Two systems can share an equation.

Layer (in Walrus: *block*). One step of the network. Walrus has 40 of them, all the same shape.

Patch, token. The model cuts the field into small squares. Each becomes a list of numbers that travels through the layers.

Neuron, activation. Inside each layer, a small network (the MLP) computes many numbers per patch. Each of those is a neuron; its value for one patch is its activation.

Direction. A fixed mix of neurons. A PDE's *directions* in a layer are the mixes along which its activations vary most (principal components). Two PDEs can use the same neurons in different mixes, so directions say more than neurons.

Removal. Changing the inside of the model while it runs (switching off neurons, or taking out directions) and measuring how much worse each PDE gets. This tests whether the model *needs* a part, not only whether it is active.

Probe, readout. A simple formula (a weighted sum of activations) trained to read a quantity, like kinetic energy, out of a layer. If a simple formula can do it, the quantity is in there.

3 What we already know: our own model

Before turning to Walrus, we built and tested the whole toolkit on a model of our own. It already gives a clear first answer to Part A, and it taught us what to watch out for.

3.1 The setup

We trained one model, *joint2*, with Poseidon-T's design [2] (a transformer shaped like a U, 32 layers, about 21 million parameters) and Poseidon's training recipe, from scratch, on 8 datasets from Poseidon's collection: Poisson and Helmholtz (elliptic), Allen–Cahn (parabolic), two wave datasets and two compressible Euler datasets (hyperbolic), and incompressible Navier–Stokes (mixed). It predicts all eight well (test errors 0.9% to 11.8%). Then we froze it and looked inside.

3.2 Four tests

1. **Neurons.** Which neurons does each PDE use? *Result:* every PDE uses almost all of them. At most 0.8% of any layer is used by one type only. Counted neuron by neuron, everything is shared.

2. **Directions.** Per PDE and layer, the directions that hold 90% of its activity (principal components), and the overlap between two PDEs' directions [12, 11]:

$$\varphi(d, e) = \frac{\|U_d^\top U_e\|_F^2}{\min(k_d,\, k_e)}$$

   where U_d holds PDE d's k_d directions as columns (each of length 1). $\varphi = 1$: the same directions; $\varphi = 0$: at right angles. *Result:* at the start and end of the model the PDEs overlap (about 0.6); in the middle, where the solving happens, much less (0.14–0.22) (Figure 3, left). The middle works like several models side by side.

3. **Removal.** Take one PDE's directions out of the middle layers while the model runs (after [13]), and measure every PDE's error:

$$a' = a - U_d U_d^\top (a - \mu)$$

   where a is a patch's activations and μ the average of the PDE that patch belongs to, so only variation along U_d is removed. Control: random directions of the same number. *Result:* Poisson, Helmholtz and Euler each break mostly when their *own* directions are removed (Figure 3, right). The two datasets of one equation hurt each other.

4. **Paths.** Which neurons does each PDE's prediction depend on? Importance of neuron i for PDE d: activation times gradient [14], summed over patches; a PDE's top 10% is its path. *Result:* two PDEs share about 20% of their paths (two unrelated lists: 10%; the same model with random weights: 44%). Only the two wave datasets

share theirs (71%).

And a fifth, **pruning**: deleting neurons outright. Deleting one PDE's top neurons hurts mostly that PDE (for Poisson, Helmholtz and Euler); deleting the least important 90% of the middle leaves Allen–Cahn, the waves and Navier–Stokes almost untouched, while Poisson, Helmholtz and Euler break (Figure 4).

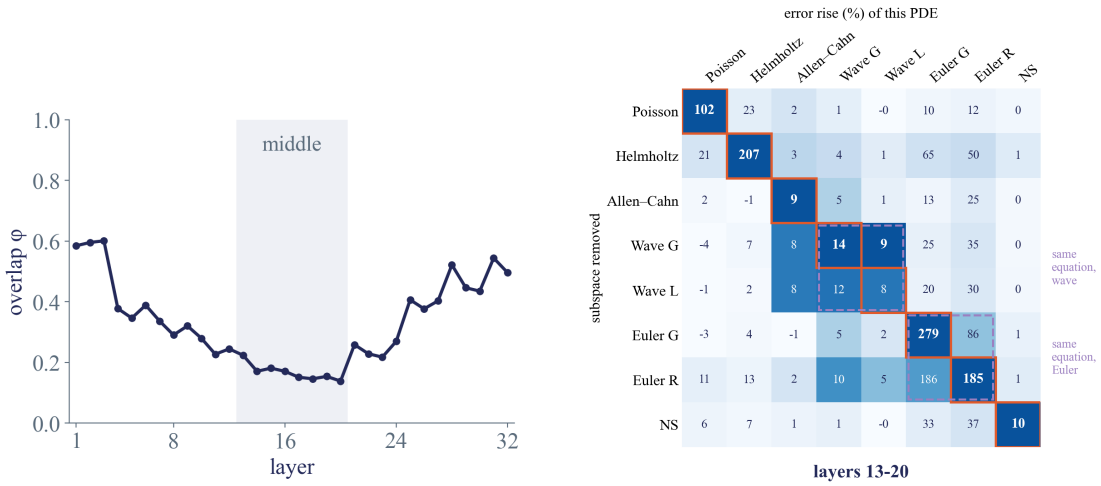


Figure 3: joint2, real results. Left: overlap of directions per layer, averaged over all 28 pairs of PDEs. The middle (grey) is where the PDEs share least. Right: removal in layers 13–20. Row: whose directions are removed; column: whose error rises (% of its normal error). Read per column: the darkest cell is the removal that hurts that PDE most; the orange boxes mark the PDE's own directions.

3.3 What it says, and what it does not

In joint2: the same neurons, used mostly separately per equation in the middle. Sharing follows the *equation* (the two waves, the two Euler sets), not the PDE *type*: two different hyperbolic equations (wave and Euler) overlap no more than an elliptic and a hyperbolic one (Figure 5).

Why this is not the end of the story.

- **Small model, few PDEs.** 21 million parameters, 8 datasets, one parabolic. Real foundation models are 50 times bigger and trained on far more physics. Sharing could appear only at that scale.

- **Room to spare.** All 8 PDEs together use only 316–761 of the 1,536 possible directions in a middle layer (Figure 6). Keeping them apart cost the model nothing. Whether they would share *if they had to* is open.

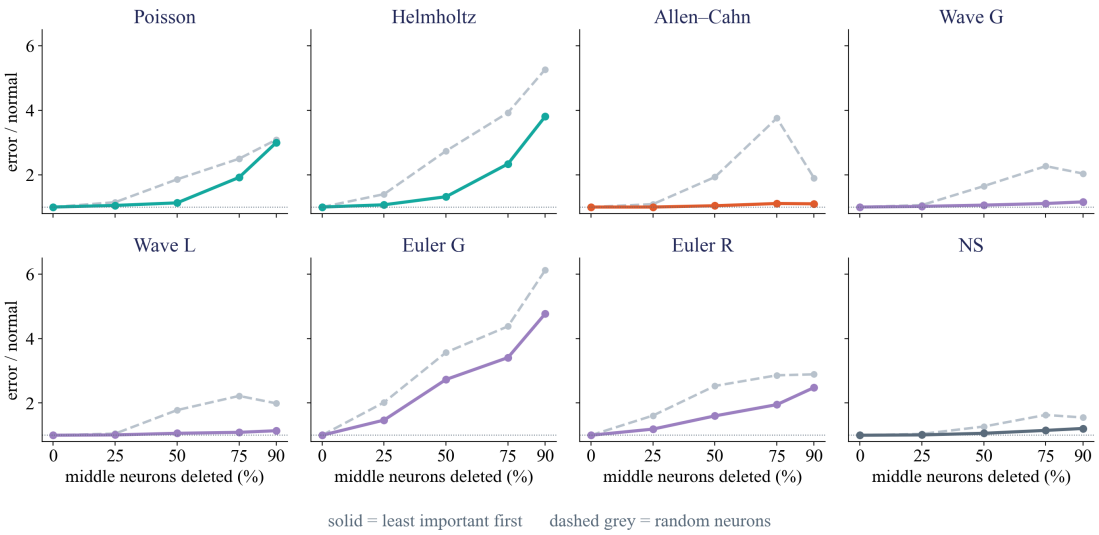


Figure 4: joint2, real results. Deleting more and more of the middle, least important neurons first (solid) or random ones (dashed grey). Some PDEs barely need the middle; Poisson, Helmholtz and Euler do.

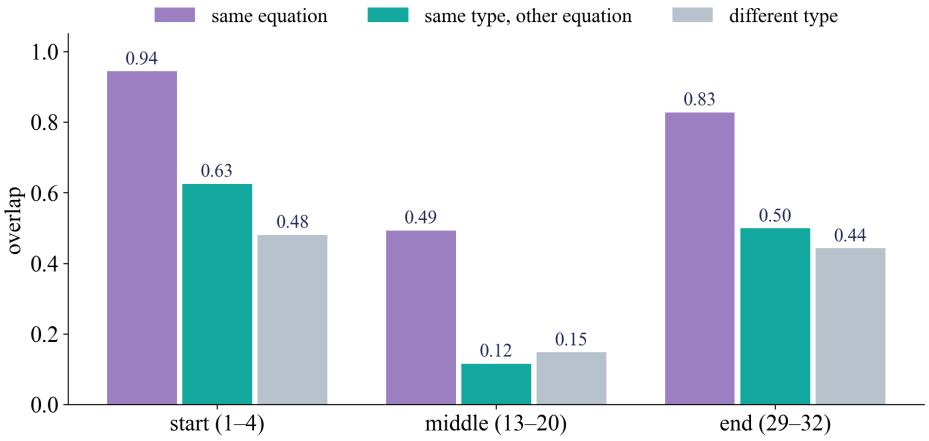


Figure 5: joint2, real results. Overlap of directions by kind of pair. In the middle, two datasets of the *same equation* overlap (0.49), but two equations of the *same type* overlap no more than two of different types (0.12 vs 0.15).

- **The U shape.** Poseidon's skips carry information around the middle. Some PDEs (Allen–Cahn, waves, Navier–Stokes) barely needed the middle at all, which made several results hard to read.

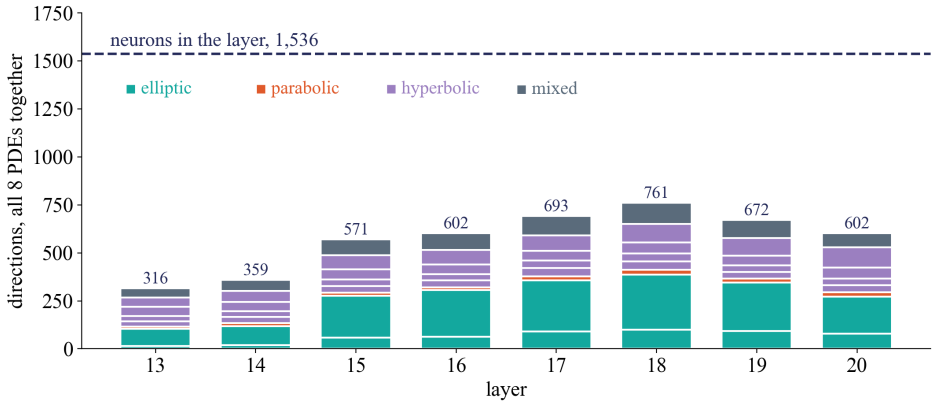


Figure 6: joint2, real results. Directions needed by all 8 PDEs together (stacked, coloured by type) per middle layer, against the 1,536 neurons available. There is plenty of room for every PDE to have its own corner.

3.4 Lessons we take along

1. **Neurons alone mislead.** Everything looks shared at the level of neurons. Directions and paths are needed.

2. **Read removal results per column.** Some PDEs are fragile (Euler breaks from almost anything, even random removals), so a row can look large for the wrong reason. Per column: which removal hurts *this* PDE most?

3. **Always a control.** Random weights for the measures, random directions or neurons for removal. The trained model must differ from them.

4. **Removing variation is not deleting.** Taking out a PDE's directions keeps each neuron's average level; deleting a neuron sets it to 0. The two gave very different answers for Allen–Cahn and the waves (they rely on a few very active neurons staying on).

5. **Do not change the design to test it.** To squeeze the model, we first removed Poseidon's skips. But then the result is about a different model. Squeezing should keep the design and only make it smaller (fewer neurons).

4 Related work

Looking inside neural networks started with language models; it reached physics foundation models in 2025–2026. A few studies look inside Walrus or Poseidon. None of them asks how much of the model different PDEs share.

Study	What they did	What is left open
Physics Steering [5] (Walrus)	A "vortex" direction found in shear flow, added to the last layer, also adds swirls in other systems.	a few hand-picked concepts, one layer, no random control; how much is shared stays open
Sparse probes and murky physics [6] (Walrus)	Sparse autoencoder features in one layer, on shear flow. Some features recur, but do not map cleanly onto known physics.	one system; not across PDEs
Liao & Cao [7] (Poseidon)	Energy can be read out of Poseidon, but the model does not use it for its predictions.	one quantity; readable is not used
pADAM [8]	Attention patterns suggest one multi-physics model reuses and combines diffusion and advection features.	one model, attention only, no removal test
Multi-physics papers [3, 9]	*Claim* a shared representation across physical systems.	not tested inside
Vafa et al. [10]	A model that predicts orbits well has not learned Newton's law: its implied forces are nonsense.	physics of planets, not PDEs

Table 1: The closest work. Our methods come from language models (directions [11], removal [13]) and attribution (paths [14, 15]).

The gap. Physics Steering shows that *some* concepts transfer between systems in Walrus. Nobody has measured *how much* of a physics foundation model different PDEs share, *where* in the network, whether that follows the PDE types, and whether the model *needs* what is shared. And the step after that, checking whether several read-out quantities agree with each other through the laws of physics, has, as far as we found, not been done for any physics model.

5 The models

5.1 The main model: Walrus

Walrus [1] is a transformer of 1.3 billion parameters, released with its code and weights. It was trained on 19 systems from The Well [4] and FlowBench: fluids, acoustics, chemistry, astrophysics, plasma, in 2D and 3D. Given the last 6 snapshots of a system, it predicts the next one (Figure 7).

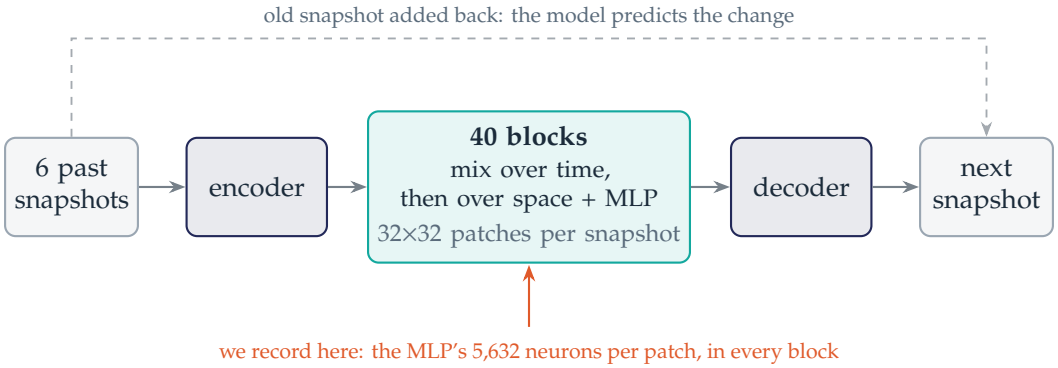


Figure 7: Walrus, simplified (sizes from [1], Table 3). All 40 blocks work on the same grid of patches (32×32 for 2D data); there are no skips around the middle. Each block mixes over time (the 6 snapshots) and over space; its MLP has 5,632 neurons per patch, which is what we record. The model predicts the change from the last snapshot.

Why Walrus.

- **A real foundation model.** Large, trained on many kinds of physics, among the best on its benchmarks [1]. Findings on it say something about physics foundation models, not about one small model.

- **Easy to look inside.** Unlike Poseidon, Walrus is not a U: all 40 blocks have the same shape and nothing goes around them. "The middle" is not a special place, every block can be compared with every other, and removal cannot be bypassed.

- **Released and studied.** Weights and code are public, and two studies looked inside it [5, 6]: we can compare with them.

- **We only need to run it.** Everything in Part A and Part B uses the frozen model: recording, removal, readouts. A 1.3-billion-parameter model fits on one GPU of the DAIC cluster for this.

What it lacks. Every Walrus system evolves in time, so there is no clean steady elliptic problem like Poisson. The Helmholtz staircase comes closest (an elliptic equation, shown

oscillating in time). Elliptic PDEs can be added later by fine-tuning (Section 7.1).

5.2 The systems we use

Eight 2D systems, chosen like our own eight: several types, and pairs of the same equation in different settings, so we can tell "same equation" from "same type" (the lesson of Figure 5).

System (The Well)	Equation	Type	Pair
acoustic scattering, inclusions	wave equation, medium with inclusions	hyperbolic	same equation
acoustic scattering, discontinuous	wave equation, medium with a jump	hyperbolic	
Euler multi-quadrants, periodic	compressible Euler (gas, shocks)	hyperbolic	same equation
Euler multi-quadrants, open	compressible Euler, open walls	hyperbolic	
Gray–Scott	reaction–diffusion (two chemicals)	parabolic	
shear flow	incompressible Navier–Stokes	mixed	same family
Rayleigh–Bénard	Navier–Stokes plus heat (convection)	mixed	
Helmholtz staircase	Helmholtz, time-harmonic waves	elliptic in space	borderline

Table 2: The eight Walrus systems for Part A, all in Walrus's pretraining data ([1], Table 6). Possible extras: shallow water on the globe (hyperbolic), active matter. 3D systems are left out to keep things comparable.

We do not need The Well's terabytes: a few dozen test trajectories per system are enough to record the inside of the model.

5.3 The lab model

Walrus cannot be retrained: it was trained on 96 H100 GPUs [1]. Some questions need retraining, such as "what if the model had less room?" or "what if it saw one system only?". For those we use a small model we train ourselves.

Two candidates, same role:

- **Poseidon-T design (joint2).** Ready now: data, training code and all four tests exist, and the first results are in. But a different design from Walrus (a U with skips).

- **A mini-Walrus.** Walrus's own code with much smaller settings, trained on a few of the systems above. Same design as the main model, so results compare directly. Costs setup and some days of training.

Plan: start the lab work on the Poseidon-T design, since it is ready. Move to a mini-Walrus if Part A shows that Walrus behaves differently from joint2, because then the design matters and the lab model should match it.

6 Part A: is it shared?

> *Do different PDEs use the same parts of Walrus, does Walrus need those parts, and does the sharing follow the PDE types or the equations?*

Part A runs the four tests of Section 3 on Walrus, step by step, then adds the lab experiments.

Step A0: get Walrus running

1. Install Walrus's code and weights on DAIC; download a few dozen test trajectories of each system in Table 2.

2. **Sanity check:** reproduce Walrus's own one-step errors on these systems [1]. If we cannot, something in our setup is wrong (normalisation, time step, field order), and nothing after this counts.

3. Make the control: the same architecture with random weights.

Step A1: record the inside

Freeze Walrus and run every system's test examples through it. In every block, record the MLP's neurons (the 5,632 numbers per patch inside its MLP, after the gate of its SwiGLU; the equivalent of what we recorded in joint2), for the newest snapshot only. Keep a random sample of patches per example, as for joint2. The same for the random-weight copy.

Watch out: Walrus reads each system through its own first projection (systems have different fields). So the first blocks can tell systems apart from the input format alone. What matters is how the tests change *deeper in*.

Step A2: neurons

Per system and block: the share of neurons that are used (average |activation| above 1% of the block's average). Neurons used by one system only, by one type only, by all. *Expected, from joint2:* almost everything used by everyone. This is the baseline that the next steps look beneath.

Step A3: directions

Per system and block: the principal components that hold 90% of its activity, and the overlap φ between every pair of systems. Three summaries:

1. **Overlap per block**, averaged over pairs, from block 1 to 40 (like Figure 3, left). Where is Walrus shared, and where split?

2. **By kind of pair:** same equation (acoustics–acoustics, Euler–Euler), same family (shear flow–Rayleigh–Bénard), same type but different equation (acoustics–Euler), different type (like Figure 5). This is the direct test of "grouped by type".

3. **Room:** the number of directions all systems need together, against the 5,632 available (like Figure 6). Does Walrus have room to spare too?

Control: random weights. Overlap in a trained block must differ from the random-weight block to mean anything.

Step A4: paths

Per system and block: the importance of every neuron for the system's prediction (activation times gradient [14]):

$$I_{d,i} = \left| \sum_{p} a_{p,i} \frac{\partial F_d}{\partial a_{p,i}} \right|$$

where $a_{p,i}$ is neuron i's activation in patch p, and F_d measures system d's prediction. A system's top 10% most important neurons is its path through that block. Two systems share a share of their paths; two unrelated lists share 10%, the random-weight model sets the other reference. This needs gradients, so it is the most expensive step, but still one run per system.

Step A5: removal and pruning

The step that tests whether the model *needs* what it shares.

1. **Remove a system's directions** in a range of blocks while Walrus runs, and measure every system's error. Read per column. Control: random directions.

2. **Delete a system's top neurons** (its path), and measure every system's error. Control: random neurons.

3. **Delete more and more**, least important first: which systems break first?

Walrus has no skips, so a removal cannot be bypassed: one of the problems we had with joint2 is gone. Both kinds of removal (directions, which keep the average level, and deleting, which does not) are reported, because they can differ.

Step A6: the lab experiments

What Walrus cannot tell us: *does it share when it has to?* On the lab model, keeping its design and only changing its size (fewer neurons per layer):

1. **Squeeze.** Train with fewer and fewer neurons (for the Poseidon-T design: 1,536, 384, 96, 24 in the middle layers), for system A alone, system B alone, and A and B together. Find the smallest size that still works for each. If together needs about the sum of the two, they are separate; if about the bigger one, they share (Figure 8). A natural first pair: Poisson and Euler, two different types that both really use the middle.

2. **Does sharing appear?** Run the direction test on each squeezed model. If overlap rises as the room shrinks, the PDEs *can* share, they just do not when they do not have to.

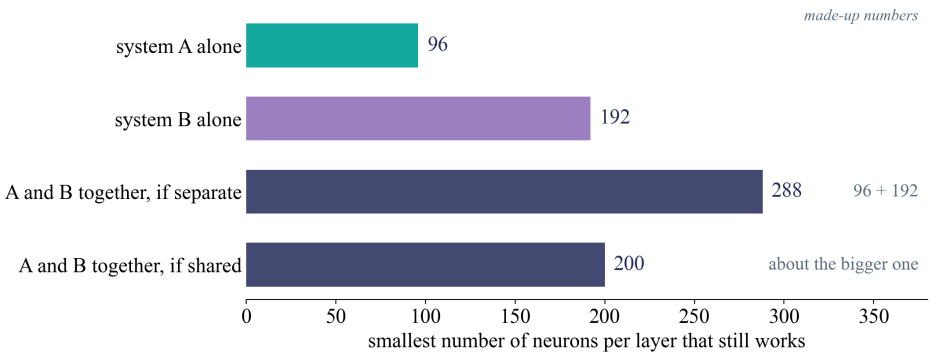


Figure 8: How the squeeze is read (made-up numbers). The smallest model that still works for each system alone, and for both together. Needing the sum means two separate things in one box; fitting in about the bigger one means they share.

Every run trains from scratch with the full recipe, so this is the expensive part of Part A (about a day per run on DAIC for the Poseidon-T design; fewer neurons train faster).

What Part A can give

If we see	It means
High overlap between types deep in Walrus, shared paths, removals that hurt several types	shared magic: story 1, and one big model is justified. On to Part B, in those blocks.
Overlap within a type clearly above between types	grouped by type: the textbook classification exists inside. On to Part B per type.
Sharing only within an equation (as in joint2)	several models in one box, organised by equation: story 2, now for a large model. The case for one big model is weak.
Shared in some blocks, split in others	a map of where the model shares: Part B on the shared blocks only.
Lab: together fits in the bigger one; overlap rises when squeezed	the PDEs *can* share; separation in large models is a luxury of room.

Every row is a finding. Since joint2 showed story 2, finding story 1 in Walrus would be the more surprising result, and finding story 2 again would show it is not a small-model accident.

7 Part B: is what is shared physics?

> *Where Part A finds sharing: does the shared part hold physical quantities, do they agree with each other the way the laws of physics say, and does the model use them?*

Part B starts only after Part A, and only in the blocks and systems where Part A found sharing. If Part A finds every system in its own corner, Part B shrinks to one system, as a smaller follow-up.

Three properties, tested in this order. Each can stop the line.

1. Readable. A simple formula can read physical quantities out of the shared part.

2. Coherent. Those read-out quantities agree with each other, the way the laws of physics say they must.

3. Used. The model relies on them: changing them inside changes its prediction the way physics says.

Others have tested 1 (and recently 3) for single quantities [7, 5]. Property 2 is new.

Step B1: readable

How. Take the activations of one block. Fit a *linear* readout: a weighted sum of the activations that predicts a physical quantity at every patch. Train it on some simulation runs, test it on others (snapshots of one run look alike, so mixing them would let the readout memorise).

Why only linear. A weighted sum cannot multiply. If a quantity needs multiplying fields together (kinetic energy $\frac{1}{2}\rho|u|^2$ needs speed times speed), a linear readout can only find it if the model has already computed it. So we choose *hard* quantities: ones that are not a weighted sum of the input. Easy ones prove nothing.

System	Hard quantities (never given to Walrus)	Shared with
compressible Euler	kinetic energy $\frac{1}{2}\rho\|u\|^2$; total energy; Mach number	kinetic energy: all flows
shear flow	kinetic energy $\frac{1}{2}\|u\|^2$; how fast the flow is sheared $\|\nabla u\|^2$	kinetic energy
Rayleigh–Bénard	kinetic energy; heat carried up by the flow $w \cdot b$	kinetic energy
acoustics	acoustic energy (pressure squared plus speed squared)	energy-like
Gray–Scott	reaction rate AB^2	—

Table 3: Candidate quantities. All need multiplying fields, so a linear readout of the raw input cannot produce them. (w: upward velocity, b: temperature, A, B: the two chemicals.)

The bridge to Part A: one readout for many systems. Kinetic energy exists in Euler, shear flow and Rayleigh–Bénard. Train the kinetic-energy readout on Euler, and test it, unchanged, on shear flow and Rayleigh–Bénard. If one direction reads kinetic energy in all three, the model has one shared concept of it. This is where "shared" (Part A) and "physics" (Part B) meet.

Controls.

- **The input itself:** a readout straight from the input fields. If it already works, the quantity is too easy.
- **Random weights:** the same readout on the random-weight Walrus.

- **Other blocks:** the same readout in every block, to see where the quantity appears.

Step B2: coherent

The idea. Imagine two models whose readouts are all about equally accurate.

- The first has a separate shortcut per quantity. Each readout looks fine, but their errors are unrelated, so together they break physics.
- The second has one picture of the world and reads everything from it. When it is wrong, everything is wrong in the same way: it describes one slightly wrong world in which the laws still hold.

Single readouts cannot tell these apart. Whether the errors hang together can (Figure 9).



Figure 9: How we see whether errors hang together (sketch). Each dot is one example: the error of readout 1 against that of readout 2. Left: the errors go together. Right: the same errors, each paired with a random other example; the link is gone.

The score, fixed before we measure. Put the readouts into a law of physics and measure how far off the law is. Then the same with the errors shuffled between examples, which breaks any link on purpose:

$$C = \frac{\text{how far off the law, with shuffled errors}}{\text{how far off the law, with the real readouts}}$$

$C \approx 1$: the real errors break the law as much as unrelated errors, so there is no shared picture. $C \gg 1$: the errors fit the law much better than chance, so they hang together.

Two laws to start with.

1. **Energy in a gas (Euler).** Total energy is internal energy plus kinetic energy:

$E = \dfrac{p}{\gamma - 1} + \frac{1}{2}\rho|u|^2$, with p the pressure and γ a fixed number of the gas. Read out E and the kinetic energy separately; does their difference match the pressure term? No derivatives needed.

2. **Heat budget of a slab (Rayleigh–Bénard).** Take a horizontal band of the fluid. The heat in it changes only through its top and bottom edges: conducted, and carried by the flow ($w \cdot b$, read out). The change itself comes from Walrus's own prediction. Does the read-out transport add up to the change the model predicts?

Slopes of read-out fields are noisy, so we never take them directly: the laws are checked as averages over smooth regions, with every slope moved onto a smooth bump we draw ourselves.

The comparison that counts: trained Walrus against random-weight Walrus. Readouts from one block share their input, so even random weights give some linked errors. Walrus must clearly beat that.

Step B3: used

A quantity can be readable without the model using it [7]. So we change it inside and watch what follows.

1. Take the readout's direction (for example "more kinetic energy", or "more heat carried up").

2. Push the block's activations a little along it while Walrus runs, as in Physics Steering [5].

3. Physics says what should follow: more heat carried up means the lower slab cools and the upper slab warms faster. Do the other readouts and Walrus's prediction follow?

4. And the reverse: take the direction out (removal, as in Part A). Does the prediction lose exactly that physics?

What Part B can give

If we see	It means
Readable, coherent and used, with one readout across systems	a shared picture of the physics, and the model relies on it. The strongest possible outcome.
Readable, but the errors do not hang together	separate shortcuts per quantity. Also a finding: probing one quantity at a time overstates what a model understands.
Readable and coherent, but not used	the picture is there, but the predictions run on something else (as Liao & Cao found for energy in Poseidon).
Not readable	the quantity is not in there, at least not in a simple form.

7.1 Later: elliptic PDEs

Walrus has no steady elliptic system. Two ways to bring one in, after Parts A and B:

- **Look only:** run our Poisson and Helmholtz data through Walrus unchanged (fed as a single step, as Poseidon does). Its predictions will be poor, but the activations still show whether Walrus places them near something it knows. Cheap, weak evidence.

- **Fine-tune:** teach Walrus Poisson and Helmholtz with a small budget, then repeat Part A. Does Poisson get a fresh corner, or move in next to an existing system? This answers a slightly different question: how the model *fits in* something new, not what it learned in pretraining.

8 Order

From cheap to expensive. Each step can change the next.

Step	What	Where	Status
0	The tests built and checked on joint2 (neurons, directions, paths, removal, pruning)	DAIC + Mac	done
1	Read the two Walrus interpretability papers [5, 6] in full; settle the systems	Mac	next
2	A0: Walrus on DAIC, test data for 8 systems, reproduce its errors, random-weight copy	DAIC	
3	A1–A3: record the inside; neurons and directions; random control	DAIC + Mac	
4	A4–A5: paths, removal, pruning	DAIC	
5	A6 in parallel: lab squeeze on the Poseidon-T design (fewer neurons), one system alone vs two together	DAIC	
6	**Decision point** with supervisor: shared or not, where, and which lab model		
7	B1: readouts in the shared blocks, including one readout across systems	DAIC + Mac	
8	B2: the coherence score on the two laws	Mac	
9	B3: steering and removal along the readouts	DAIC	
10	Optional: elliptic PDEs (look only, then fine-tune); a mini-Walrus lab model	DAIC	

Steps 2–4 reuse our existing scripts; the work is mostly adapting them to Walrus's layout (6 snapshots, 40 blocks, its own data format). Steps 2–6 can be the first paper on their own; Part B builds on it.

9 Risks, and what we do about them

1. **Getting Walrus to run correctly takes time.** New code, its own data handling and normalisation. *What we do:* the sanity check in A0. Nothing counts until we reproduce its published errors.

2. **The input format separates systems on its own.** Systems with different fields enter through different projections. *What we do:* judge sharing by how it changes deeper in, and against the random-weight model.

3. **One neuron, many meanings.** In large models one neuron often carries several things (superposition). *What we do:* that is why directions and paths are the main tests, not single neurons. A sparse autoencoder, as in [6], is a possible extra.

4. **No elliptic system.** *What we do:* test the type question on parabolic, hyperbolic and mixed; add elliptic later (Section 7.1).

5. **Lab model and Walrus differ in design.** *What we do:* start with Poseidon-T because it is ready; switch to a mini-Walrus if the two disagree.

6. **The answer is "separate" again.** Not a failure: it is story 2, now for a large model, and with the lab results it says whether separation is a choice (room) or a necessity.

10 What comes out

- **A map of sharing inside a physics foundation model:** per block, per pair of systems, by type and by equation, with removal tests showing what the model needs. For Walrus, and for our lab model as a second design.

- **What the inside says about one big model for all physics:** shared magic that justifies it, or several models in one box.

- **If shared: whether it is physics.** Readable, coherent, used, and whether one readout works across systems.

- **A reusable toolkit** (neurons, directions, paths, removal, coherence score) for any physics foundation model.

In one line. Part A says whether physics foundation models are one model or several in a box; Part B says, where they are one, whether what holds them together is physics.

References

[1] M. McCabe et al. *Walrus: A Cross-domain Foundation Model for Continuum Dynamics.* ICML 2026. arXiv:2511.15684

[2] M. Herde et al. *Poseidon: Efficient Foundation Models for PDEs.* NeurIPS 2024. arXiv:2405.19101

[3] M. McCabe et al. *Multiple Physics Pretraining for Physical Surrogate Models.* 2023. arXiv:2310.02994

[4] R. Ohana et al. *The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning.* 2025. arXiv:2412.00568

[5] R. Fear, P. Mukhopadhyay, M. McCabe, A. Bietti, M. Cranmer. *Physics Steering: Causal Control of Cross-Domain Concepts in a Physics Foundation Model.* NeurIPS 2025. arXiv:2511.20798

[6] Rosenfeld, Sonnewald. *Sparse Probes and Murky Physics: a Case Study of Interpretability Challenges in a Foundation Model for Continuum Dynamics.* 2026. arXiv:2606.11657

[7] C.-T. Liao, X. Cao. *The Objective Decides: When a Learned Dynamics Model Uses a Conserved Quantity.* 2026. arXiv:2607.03728

[8] *pADAM: A Plug-and-Play All-in-One Diffusion Architecture for Multi-Physics Learning.* 2026. arXiv:2603.16757

[9] *Unsupervised Adaptation of PDE Foundation Models.* 2026. arXiv:2608.07053

[10] K. Vafa, P. G. Chang, A. Rambachan, S. Mullainathan. *What Has a Foundation Model Found? Using Inductive Bias to Probe for World Models.* ICML 2025. arXiv:2507.06952

[11] T. A. Chang, Z. Tu, B. K. Bergen. *The Geometry of Multilingual Language Model Representations.* EMNLP 2022.

[12] E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models.* 2021 (Eq. 4, subspace similarity). arXiv:2106.09685

[13] Saha, Bergen, Khosla. *Neural Networks Are More Modular Than Single Neurons Suggest.* NeurIPS 2026.

[14] K. Dhamdhere, M. Sundararajan, Q. Yan. *How Important Is a Neuron?* ICLR 2019. arXiv:1805.12233

[15] M. Dreyer, E. Purelku, J. Vielhaben, W. Samek, S. Lapuschkin. *PURE: Turning Polysemantic Neurons Into Pure Features by Identifying Relevant Circuits.* 2024. arXiv:2404.06453